CIS 3370 · INTRUSION DETECTION SYSTEMS · LAB A OF 3

The stack, safe access, and Filebeat

Before you ship a single log you need to know three things: where every file lives and which ones you may edit, how to talk to Elasticsearch without typing a password into a command, and how to give a shipper an account that can write and do nothing else. Then you install Filebeat on `client-01` and send it two kinds of evidence: a plain file, and the system's own login records. Every change is shown three ways, in the Kibana screens, in the Dev Tools Console, and with `curl` in a terminal. Pick the one you like; the result is the same. Every number and screenshot on this page comes from running these steps on the class stack.

1. Where things live
2. Three ways in
3. No passwords on screen
4. A user that can only write
5. Install Filebeat
6. A file as a stream
7. The system module

0 Before you start

You need the class docker-elk stack running with its five services: `elasticsearch`, `logstash`, `kibana`, `client-01` and `zeek-lab`. The compose file is Appendix A. You also need `cis3370-generate-bundle.zip` from Canvas, the Lecture 6 capture and baselines, made on our own lab machine.

This lab talks about three kinds of terminal, and every command block says which one it belongs in:

LABEL	WHERE YOU TYPE IT
HOST	The machine Docker runs on. On Windows with WSL that is your Ubuntu terminal, not PowerShell. On a Mac or Linux it is your normal terminal. Always start in your `docker-elk` folder.
CLIENT-01	A shell inside the container, opened from the host with `docker compose exec client-01 bash`. You are root, and the prompt starts in `/home/ubuntu`.
ZEEK-LAB	The same for the other lab machine: `docker compose exec zeek-lab bash`.

Copy the bundle into the `client-01` data folder.

HOST On a Mac or Linux:

```
cp ~/Downloads/cis3370-generate-bundle.zip ~/docker-elk/client-01/data/
```

HOST On Windows, WSL sees your C: drive under `/mnt/c`. Put your Windows user name in place of `YOU`:

```
cp /mnt/c/Users/YOU/Downloads/cis3370-generate-bundle.zip \
  ~/docker-elk/client-01/data/
```

This lab was run on a Mac with Docker Desktop. Every command inside the containers is identical on Windows, Linux and macOS, because the containers are the same Ubuntu. The WSL-specific paths on this page come from Microsoft's WSL documentation.

1 Where things live, and what you can edit

HOST Ask Docker what is running, which ports each container publishes, and which folders each one mounts:

```
docker compose ps --format "table {{.Name}}\t{{.Ports}}"
docker compose ps --format "table {{.Name}}\t{{.Mounts}}" --no-trunc
```

The field is `.Name`. Typing `.Names`, the field for `docker ps`, prints `<no value>` for every name. The class stack printed this, with the home folder shortened to `you`. On WSL the paths start with `/home/you` instead of `/host_mnt/Users/you`.

```
NAME                        PORTS
docker-elk-client-01-1
docker-elk-elasticsearch-1  0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 0.0.0.0:9300-
>9300/tcp, ...
docker-elk-kibana-1         0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
docker-elk-logstash-1       0.0.0.0:5044->5044/tcp, [::]:5044->5044/tcp, 0.0.0.0:9600-
>9600/tcp, ...
docker-elk-zeek-lab-1

NAME                        <no value>
docker-elk-client-01-1      /host_mnt/Users/you/docker-elk/client-01/data
docker-elk-elasticsearch-1  /host_mnt/Users/you/docker-
elk/elasticsearch/config/elasticsearch.yml,docker-elk_elasticsearch
docker-elk-kibana-1         /host_mnt/Users/you/docker-elk/kibana/config/kibana.yml
docker-elk-logstash-1       /host_mnt/Users/you/docker-
elk/logstash/config/logstash.yml,/host_mnt/Users/you/docker-elk/logstash/pipeline
docker-elk-zeek-lab-1       /host_mnt/Users/you/docker-elk/zeek-lab/data
```

The lab machines publish no ports, so nothing outside Docker can reach them. Elasticsearch is on 9200 and Kibana on 5601. The Mounts column has no heading of its own, which is why the header row says `<no value>`.

Some mounts are read-only on purpose

The compose file mounts the stack's configuration files with `:ro`, read-only. The container can read them but not change them, so a broken or compromised container cannot rewrite its own settings. Try it.

HOST

```
docker compose exec logstash bash -c 'echo "# test" >> config/logstash.yml'
docker compose exec kibana bash -c 'touch config/kibana.yml'
```

```
bash: line 1: config/logstash.yml: Read-only file system
touch: cannot touch 'config/kibana.yml': Read-only file system
```

ON THE HOST	INSIDE THE CONTAINER	CONTAINER CAN WRITE?
`elasticsearch/config/elasticsearch.yml`	`/usr/share/elasticsearch/config/elasticsearch.yml`	No
`kibana/config/kibana.yml`	`/usr/share/kibana/config/kibana.yml`	No

`logstash/config/logstash.yml`	`/usr/share/logstash/config/logstash.yml`	No
`logstash/pipeline/`	`/usr/share/logstash/pipeline/`	No
`client-01/data/`	`/home/ubuntu/data/` in `client-01`	Yes, both ways
`zeek-lab/data/`	`/home/ubuntu/data/` in `zeek-lab`	Yes, both ways

So configuration files are edited **on the host**, then the service is restarted with `docker compose restart <service>`. Lab C does exactly that with `logstash.yml`. Three ways to edit a host file:

- **nano, in the host terminal.** Works everywhere: `nano logstash/config/logstash.yml`, save with Ctrl+O and Enter, leave with Ctrl+X.
- **VS Code.** On Windows install the *WSL* extension, then run `code .` from your `docker-elk` folder in the Ubuntu terminal. VS Code opens the Linux files directly. On a Mac or Linux, `code .` works the same way.
- **Windows Explorer.** Type `\\wsl.localhost\Ubuntu\home\YOU\docker-elk` in the address bar to browse the files from Windows. Windows editors may save lines ending in CRLF, which Linux programs misread. If you edit this way, set the editor to LF line endings first.

Inside the lab machines, only `data/` survives. Packages you install with `apt-get`, and everything under `/etc/filebeat`, live in the container. They are gone when the container is recreated, for example after `docker compose down`. Files in `data/` stay, because they are folders on your host.

2 Three ways to talk to Elasticsearch

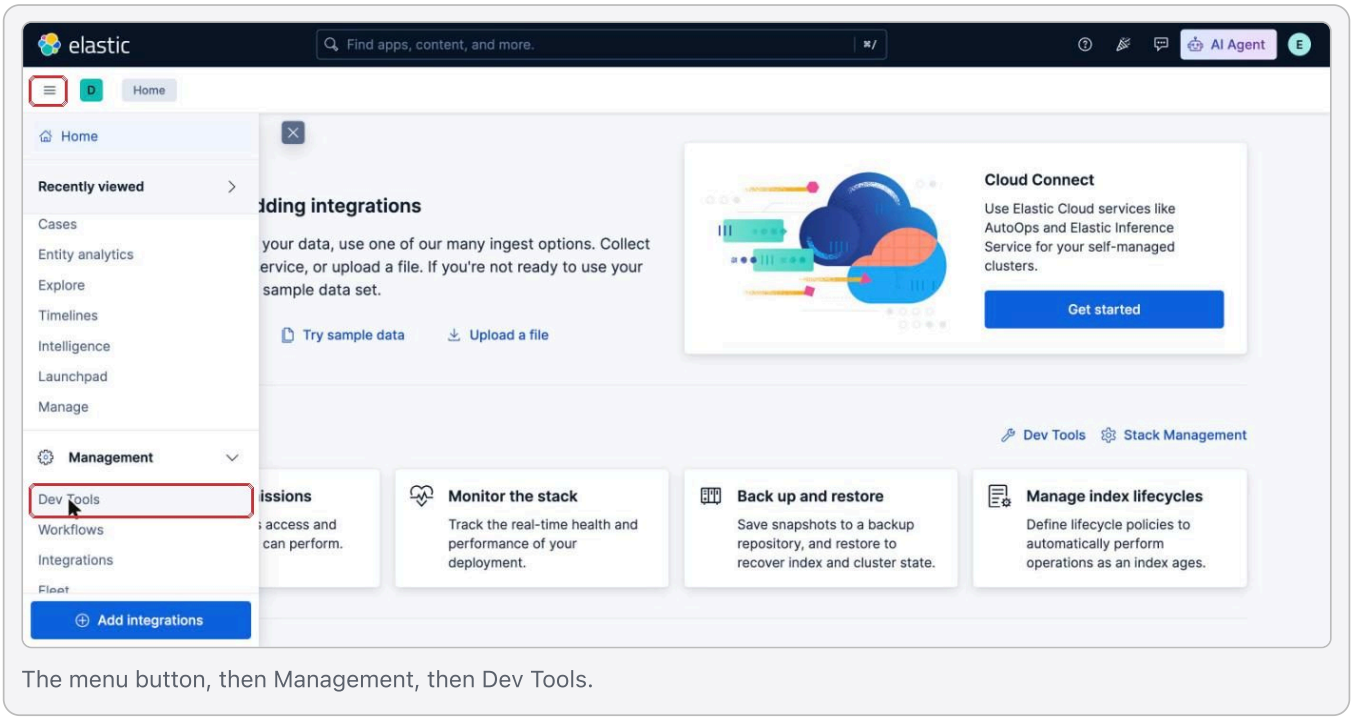
Elasticsearch is a web service. Every action is an HTTP request: a *method* that says what to do, and a *path* that says to what. Four methods cover almost everything:

METHOD	MEANS	EXAMPLE
`GET`	Read something	`GET _cluster/health`
`PUT`	Create or replace the thing at this exact name	`PUT _security/role/filebeat_publisher`
`POST`	Create something and let the server name it, or run an action	`POST _security/api_key`
`DELETE`	Remove it	`DELETE _security/api_key`

Most requests that create something carry a body in JSON, a list of `"name": value` pairs inside braces. Every answer comes back as JSON too, with an HTTP status: 200 or 201 means it worked, 401 means Elasticsearch does not know who you are, 403 means it knows and the answer is no, and 404 means not found.

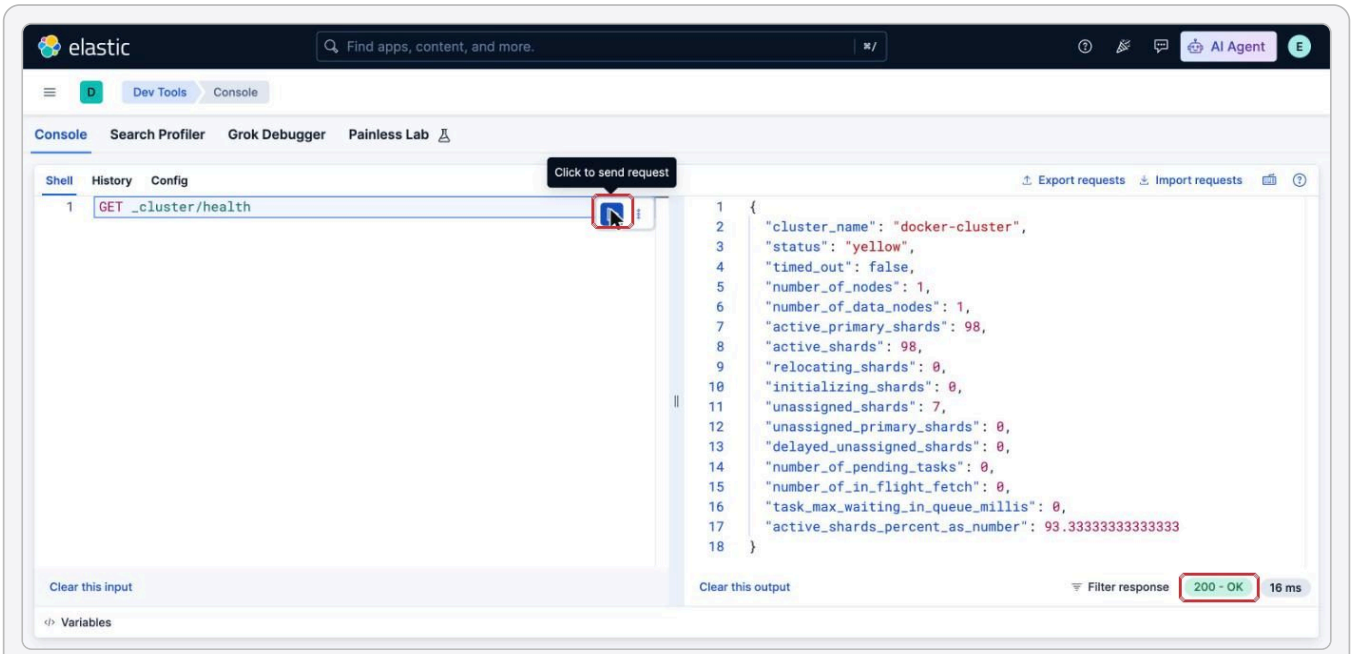
The Dev Tools Console

KIBANA Open http://localhost:5601, click the menu button (three lines, top left), scroll to **Management**, and choose **Dev Tools**. The **Console** tab opens.



The menu button, then Management, then Dev Tools.

Click **Clear this input** under the left pane, type `GET _cluster/health`, and send it with the triangle button or **Ctrl+Enter** (Cmd+Enter on a Mac). The answer appears on the right with its status, `200 – OK`.



Send the request with the triangle button. The status is at the bottom right.

`"status": "yellow"` is correct for a one-node cluster: every index is searchable, but there is no second machine to hold a copy. The Console needs no password because it uses your Kibana login. It is the easiest place to try a request.

curl, from a terminal

HOST The same request without credentials:

```
curl -s "http://localhost:9200/_cluster/health?pretty"
```

```
"reason" : "missing authentication credentials for REST request [/_cluster/health?pretty]"
... "status" : 401
```

401: Elasticsearch does not know who is asking. The old way to answer is `-u elastic:changeme`, which puts the password on your screen, in your shell history and in every script you copy it into. Section 3 replaces it.

The Kibana screens

Everything under **Menu → Management → Stack Management** is a form that sends these same requests for you. Users, roles, API keys, data views and Logstash pipelines all live there. They are the friendliest way in and the hardest to repeat exactly, which is why this lab shows all three.

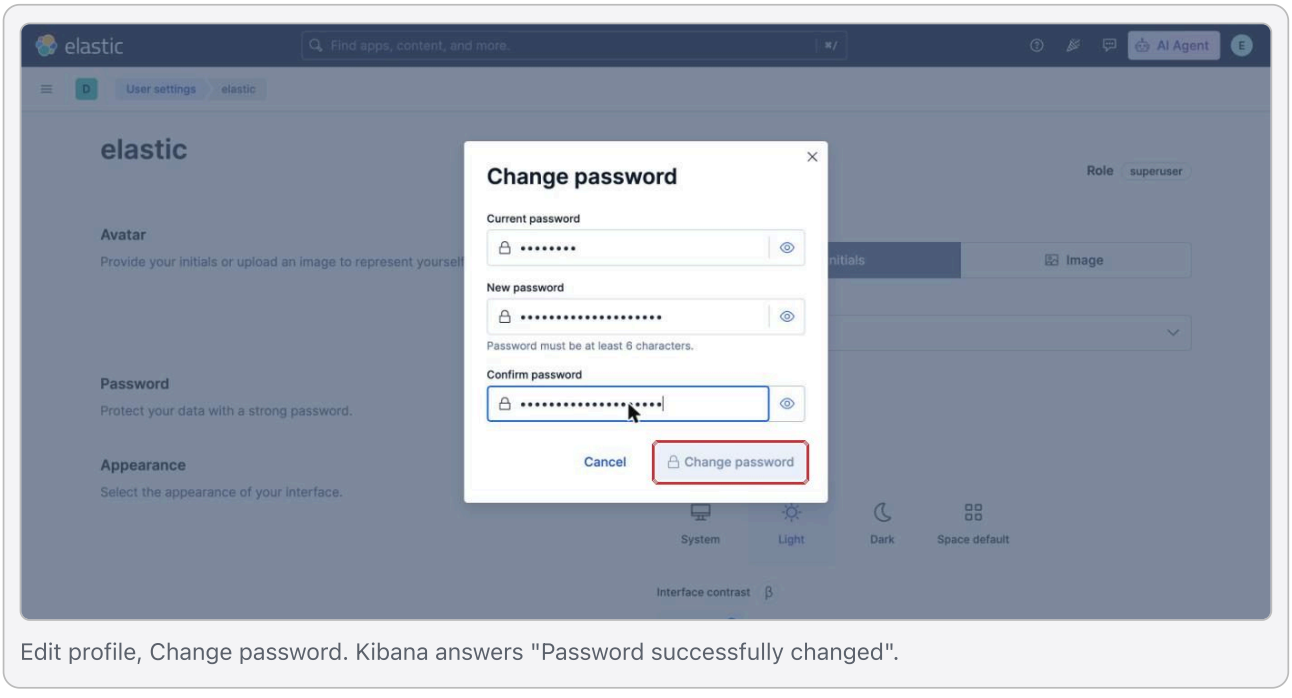
3 No passwords on screen

Change the elastic password

docker-elk starts with `changeme`, and so does everyone else's copy. Change it first.

KIBANA UI, RECOMMENDED FOR PASSWORDS

Click your initial (top right), then **Edit profile**, then **Change password**. Type the current password, then the new one twice. The dots never show what you typed.



Edit profile, Change password. Kibana answers "Password successfully changed".

CONSOLE

```
POST _security/user/elastic/_password
{
  "password": "your-new-password"
}
```

This works, but the password is now in the Console's History tab and in your browser's storage. Use the UI for passwords.

CURL

`HOST` With `-u elastic` and no password, curl asks for it and does not echo it. `read -rs` does the same for the new one.

```
read -rsp "New elastic password: " NEWPW; echo
curl -s -u elastic -X POST \
  "http://localhost:9200/_security/user/elastic/_password" \
  -H 'Content-Type: application/json' -d "{\"password\": \"$NEWPW\"}"
unset NEWPW
```

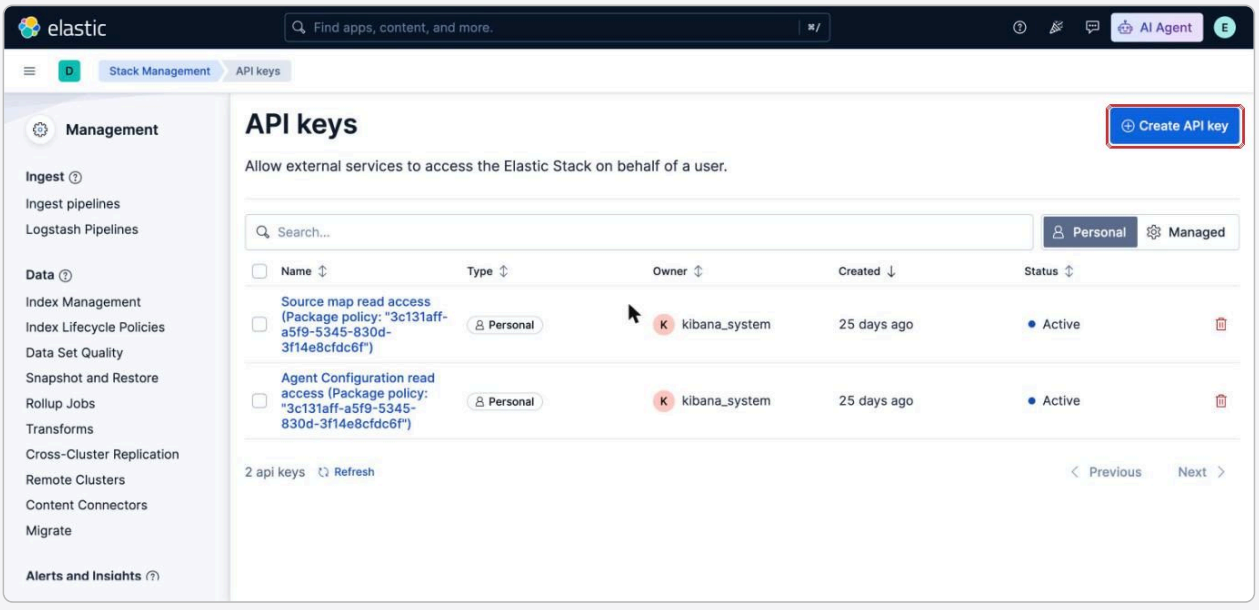
On the class stack the old password then got 401 and the new one 200. Kibana keeps you logged in.

Create an API key

An API key is a second credential for your account, made for programs. It can expire, it can be revoked on its own without touching your password, and it can be given fewer rights than you have.

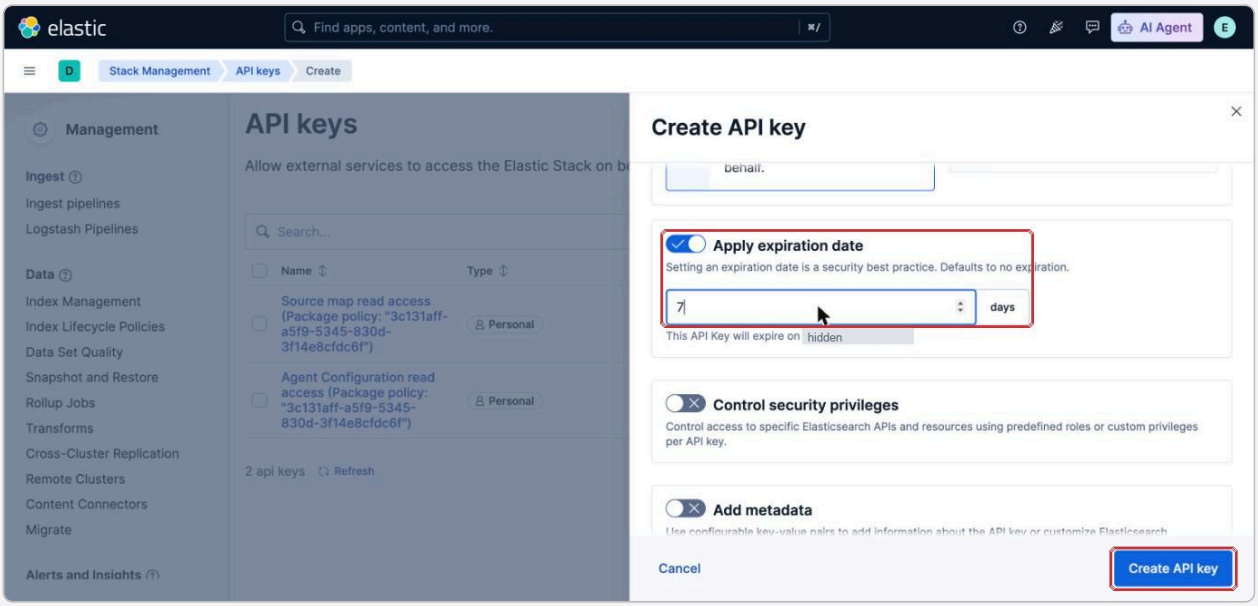
KIBANA UI

Menu → Management → Stack Management → Security → API keys, then **Create API key**.



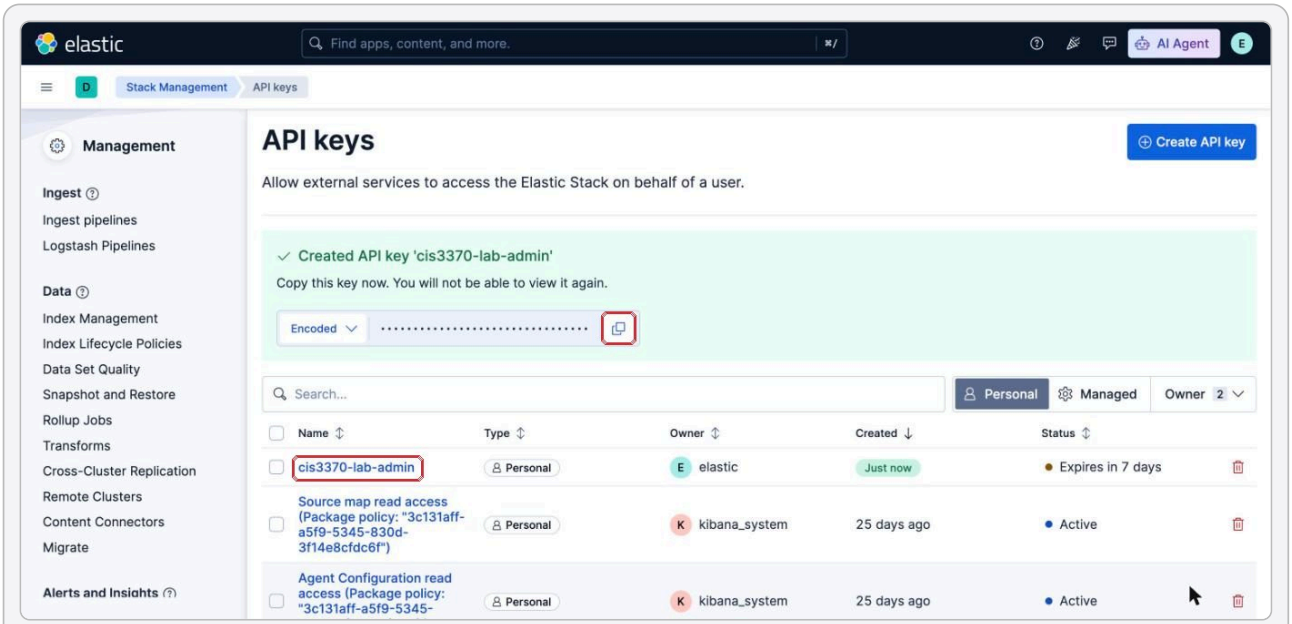
Stack Management, API keys. The two keys already there belong to Kibana itself.

Name it `cis3370-lab-admin`, turn on **Apply expiration date** and enter `7` days. Leave **Control security privileges** off: the key gets your rights, which you need for this lab.



Name, expiration in days, then Create API key.

Kibana shows the key once. Use the copy button next to the **Encoded** value. You cannot see it again; if you lose it, revoke it and make another.



The key is shown once. Copy the Encoded value with the button. It is hidden here.

CONSOLE

```
POST _security/api_key
{
  "name": "cis3370-lab-admin",
  "expiration": "7d"
}
```

The answer has `id`, `api_key` and `encoded`. Copy `encoded`.

CURL

`HOST` curl asks for the elastic password.

```
curl -s -u elastic -X POST "http://localhost:9200/_security/api_key?pretty" \
  -H 'Content-Type: application/json' \
  -d '{"name": "cis3370-lab-admin", "expiration": "7d"}'
```

Keep the key in a file, not on your screen

`HOST` Paste the Encoded value at the prompt. Nothing shows while you paste.

```
mkdir -p ~/.elastic && chmod 700 ~/.elastic
read -rsp "Paste the Encoded API key, then Enter: " KEY; echo
printf 'Authorization: ApiKey %s\n' "$KEY" > ~/.elastic/auth.hdr
unset KEY
chmod 600 ~/.elastic/auth.hdr
```

The file holds one line, an HTTP header, and only you can read it. curl reads a header from a file with `-H @file`. Set two short names once per terminal:

```
ES=http://localhost:9200
KEYFILE=@$HOME/.elastic/auth.hdr
curl -s -H "$KEYFILE" "$ES/_security/_authenticate?pretty"
```

The answer says `"username" : "elastic"` and `"authentication_type" : "api_key"`. From here on, every curl command in these labs starts with `curl -s -H "$KEYFILE"`, and no secret appears in a command, on screen or in your history. Add the two lines to `~/.bashrc` if you are tired of typing them.

An API key cannot make another API key. Using the key to run `POST _security/api_key` is refused:

```
creating derived api keys requires an explicit role descriptor that is empty (has no
privileges)
```

Create keys in the Kibana UI, in the Console, or with curl and `-u elastic`.

Revoke a key

Keys leak. Revoking one is instant and touches nothing else. On the class stack a test key got 200 on `_cluster/health`, was revoked, and then got 401: `unable to authenticate with provided credentials`.

KIBANA UI

In the API keys list, the red bin icon at the end of the row.

CONSOLE

```
DELETE _security/api_key
{
  "name": "cis3370-revoke-demo"
}
```

CURL

```
curl -s -H "$KEYFILE" -X DELETE "$ES/_security/api_key" \
  -H 'Content-Type: application/json' -d '{"name": "cis3370-revoke-demo"}'
```

4 A user that can only write

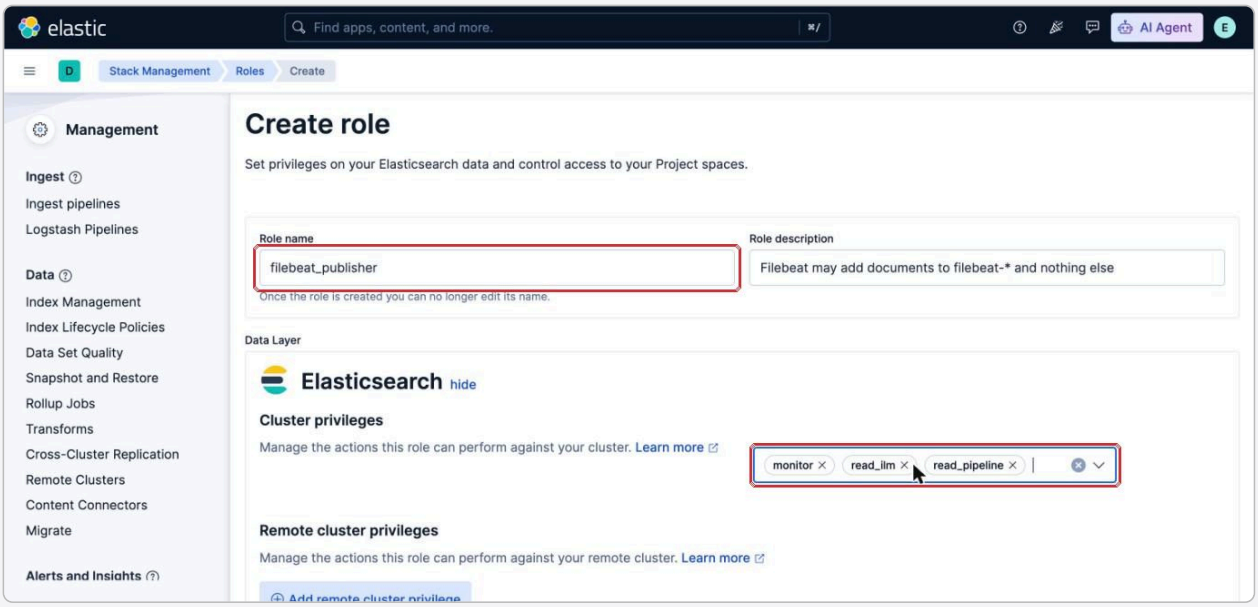
Filebeat has two jobs. **Setup** happens once: it creates the index template, the lifecycle policy and the ingest pipelines, and needs administrator rights. **Publishing** happens all day on every machine that ships logs, and only adds documents. A shipper's password sits on the machine it watches. If that machine is broken into, the attacker gets the password, so the publishing account must be able to add documents and do nothing else. Setup is done with your admin key; Filebeat's own account is `beats`.

PRIVILEGE	WHERE	WHY FILEBEAT NEEDS IT
`monitor`	cluster	Check the Elasticsearch version and license when it connects
`read_ilm`	cluster	Check that the lifecycle policy exists
`read_pipeline`	cluster	Check that a module's ingest pipeline exists
`create_doc`	`filebeat-*`	Add new documents, never change or delete them
`auto_configure`	`filebeat-*`	Create the data stream, and add a field it has never seen

The role

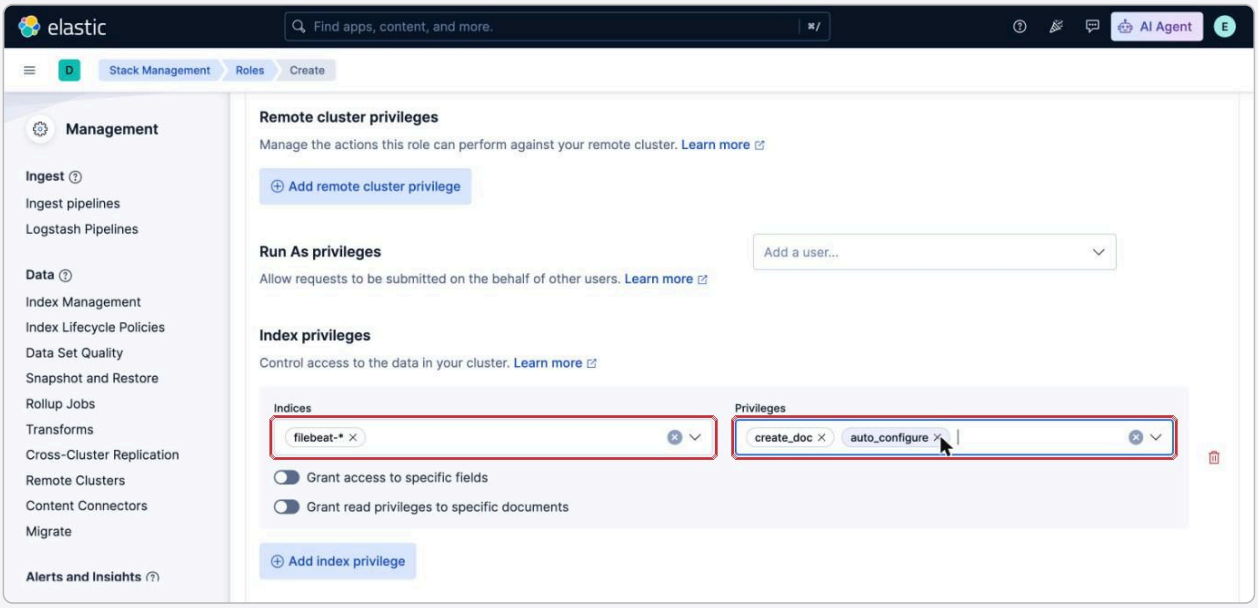
KIBANA UI

Stack Management → Security → Roles → Create role. Name it `filebeat_publisher` and add the three cluster privileges.



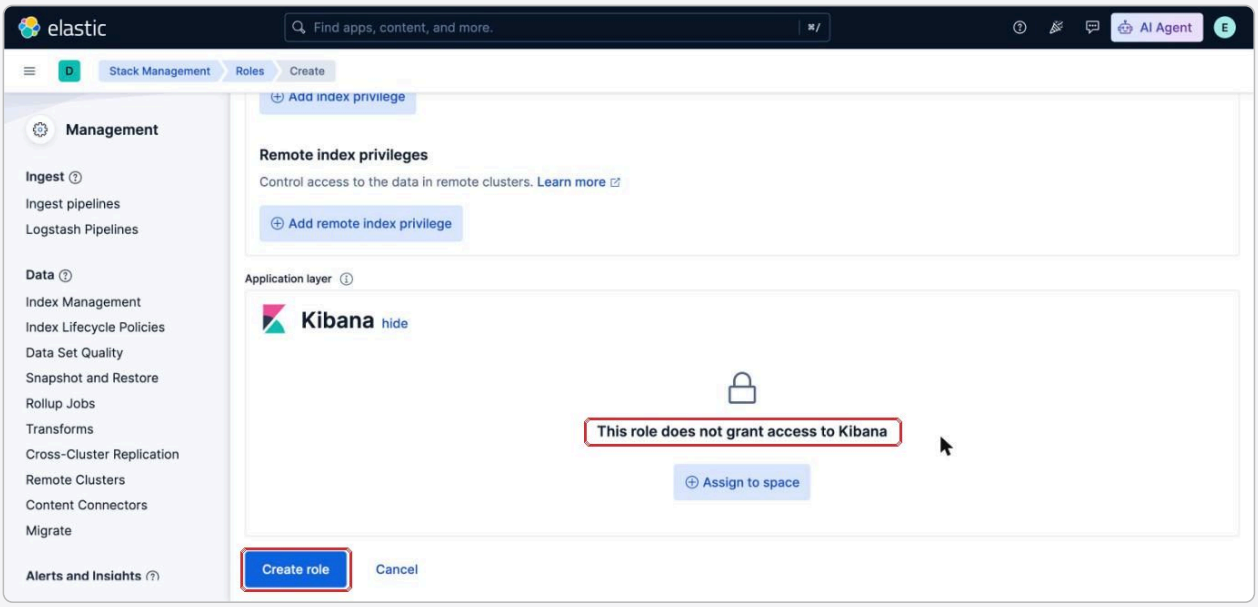
Role name, then the three cluster privileges.

Under **Index privileges**, indices `filebeat-*`, privileges `create_doc` and `auto_configure`. Type `filebeat-*` in full and press Enter.



One index privilege row: filebeat-* with create_doc and auto_configure.

At the bottom, the Kibana section says the role grants no access to Kibana. That is what you want. Click **Create role**.



No Kibana access, then Create role.

CONSOLE

```
PUT _security/role/filebeat_publisher
{
  "description": "Filebeat may add documents to filebeat-* and nothing else",
  "cluster": ["monitor", "read_ilm", "read_pipeline"],
  "indices": [
    {
      "names": ["filebeat-*"],
      "privileges": ["create_doc", "auto_configure"]
    }
  ]
}
```

CURL

HOST Write the body to a file, then send it. Long JSON is easier to check in a file than on one line.

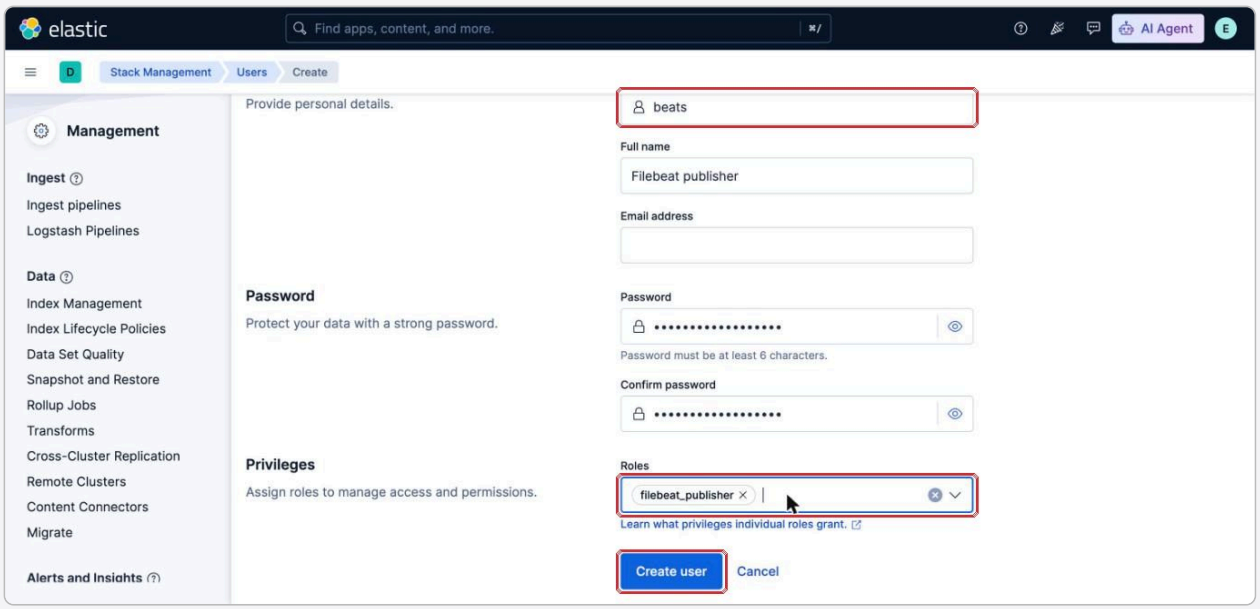
```
cat > /tmp/role.json <<'EOF'
{
  "description": "Filebeat may add documents to filebeat-* and nothing else",
  "cluster": ["monitor", "read_ilm", "read_pipeline"],
  "indices": [
    {
      "names": ["filebeat-*"],
      "privileges": ["create_doc", "auto_configure"]
    }
  ]
}
EOF
curl -s -H "$KEYFILE" -X PUT "$ES/_security/role/filebeat_publisher" \
  -H 'Content-Type: application/json' -d @/tmp/role.json
```

Expected: `{"role":{"created":true}}`.

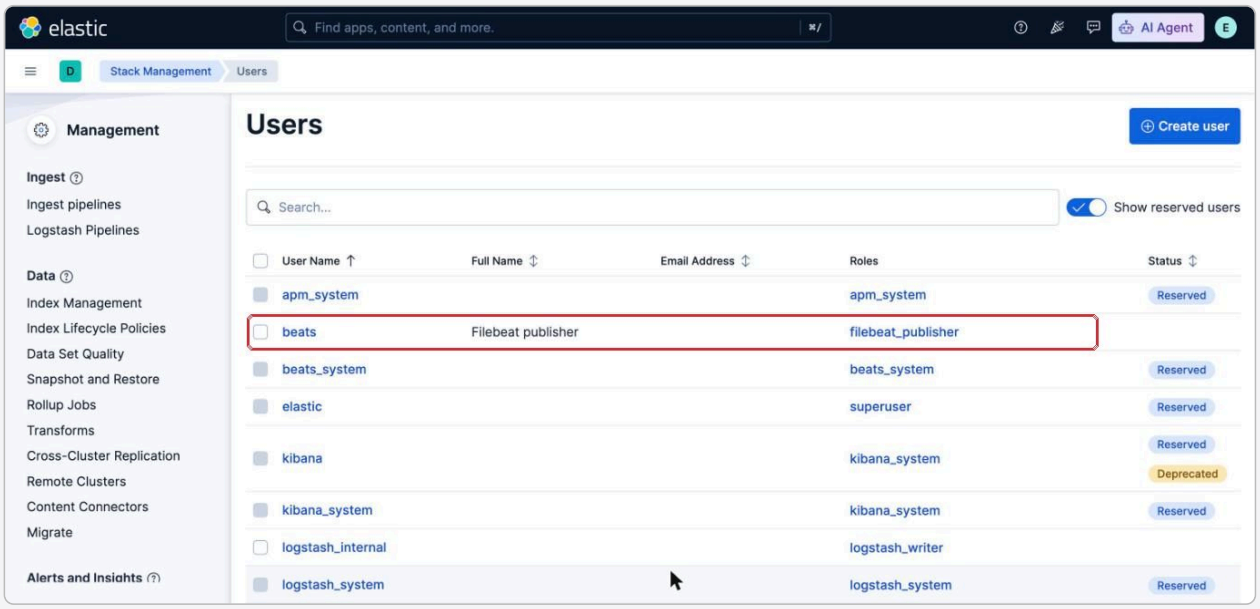
The user

KIBANA UI, RECOMMENDED FOR PASSWORDS

Stack Management → Security → Users → Create user. Username `beats`, full name `Filebeat publisher`, a password of your own, and the role `filebeat_publisher`. Then **Create user**.



Username, password twice, and the one role.



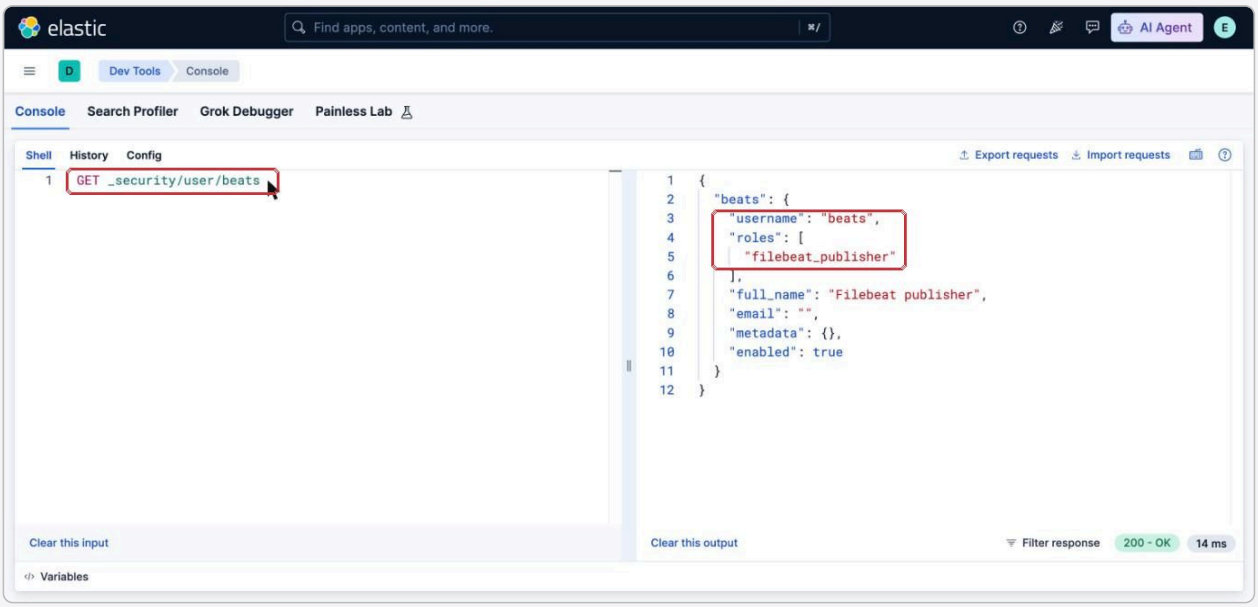
beats is listed with filebeat_publisher and no Reserved badge.

CONSOLE

```
PUT _security/user/beats
{
  "password": "choose-your-own",
  "roles": ["filebeat_publisher"],
  "full_name": "Filebeat publisher"
}
```

The password stays in the Console history. Then check the user without any secret:

```
GET _security/user/beats
```



GET _security/user/beats shows the role and enabled: true.

CURL

```
read -rsp "Password for beats: " BEATS_PW; echo
curl -s -H "$KEYFILE" -X PUT "$ES/_security/user/beats" \
  -H 'Content-Type: application/json' \
  -d "{\"password\": \"$BEATS_PW\", \"roles\": [\"filebeat_publisher\"]}"
unset BEATS_PW
```

Expected: `{"created":true}`.

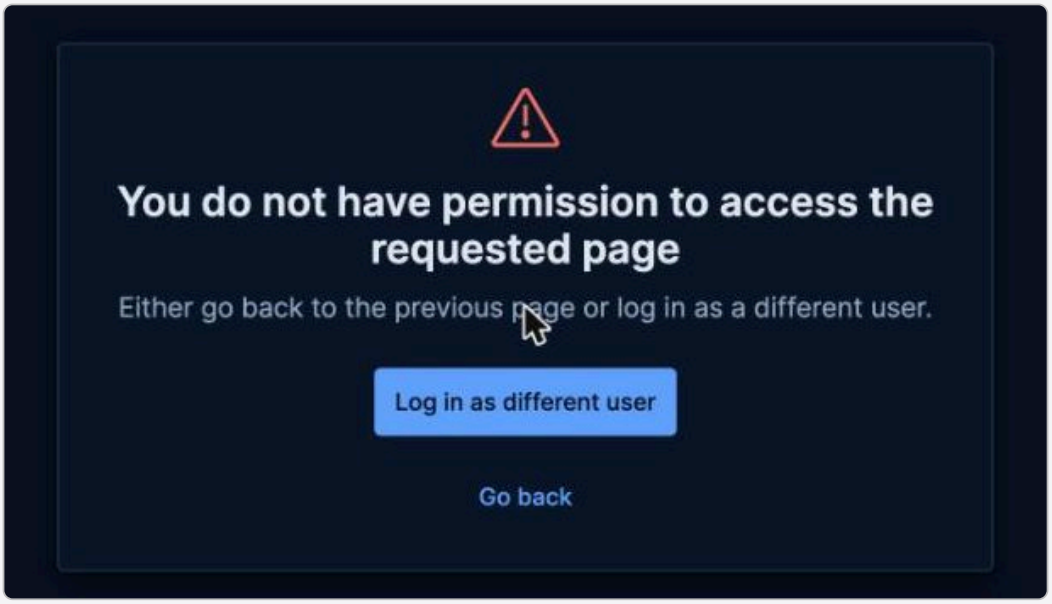
Prove the restriction

HOST Type the beats password once. Each line prints the request and its HTTP status.

```
read -rsp "Password for beats: " BEATS_PW; echo
for p in _cluster/health nsl-kdd-train/_search _security/user; do
  curl -s -o /dev/null -w "$p %{http_code}\n" -u "beats:$BEATS_PW" "$ES/$p"
done
curl -s -o /dev/null -w "write lab-probe %{http_code}\n" \
  -u "beats:$BEATS_PW" -X POST "$ES/lab-probe/_doc" \
  -H 'Content-Type: application/json' -d '{"x":1}'
unset BEATS_PW
```

```
_cluster/health 200
nsl-kdd-train/_search 403
_security/user 403
write lab-probe 403
```

It may check the cluster, and it may not read data, list users, or write anywhere but `filebeat-*`. Now log out of Kibana and log in as `beats`:



Logged in as beats, Kibana refuses every page.

Log back in as `elastic` to continue.

5 Install Filebeat on client-01

This follows Elastic's *Filebeat quick start: installation and configuration*, the DEB tab, with the version pinned to **9.5.3**, the version of the cluster. Filebeat must never be newer than Elasticsearch. `dpkg --print-architecture` picks `arm64` on an Apple silicon Mac and `amd64` on most Windows and Intel machines.

HOST

```
docker compose exec client-01 bash
```

CLIENT-01

```
apt-get update
apt-get install -y curl unzip nano
cd /tmp
ARCH=$(dpkg --print-architecture)
URL=https://artifacts.elastic.co/downloads/beats/filebeat
curl -L -O $URL/filebeat-9.5.3-${ARCH}.deb
dpkg -i filebeat-9.5.3-${ARCH}.deb
filebeat version
```

```
filebeat version 9.5.3 (arm64), libbeat 9.5.3 [...]
```

The Elastic page then says `systemctl start filebeat`. A container has no `systemd`, so here you start Filebeat yourself with `nohup filebeat -e &` and stop it with `pkill filebeat`.

Two secrets, both in the Filebeat keystore

Filebeat keeps secrets in an encrypted keystore and lets the configuration refer to them as `${NAME}`. Two go in: the beats password, which Filebeat uses all the time, and your admin key, which setup uses once and which you remove at the end of this lab.

CLIENT-01

```
filebeat keystore create
read -rsp "Password for beats: " PW; echo
printf '%s' "$PW" | filebeat keystore add ES_PWD --stdin
unset PW
```

HOST Filebeat wants an API key as `id:key`. The Encoded value in your header file is exactly that, base64-encoded, so decode it on the way in. The key never appears on screen:

```
sed 's/^Authorization: ApiKey //' ~/.elastic/auth.hdr \
  | tr -d '\n' | base64 -d \
  | docker compose exec -T client-01 filebeat keystore add ADMIN_KEY --stdin
```

CLIENT-01

```
filebeat keystore list
```

```
ES_PWD
ADMIN_KEY
```

6 A simple file, read as a stream

The simplest thing Filebeat does is read a file line by line, send every line, then keep watching and send each new line as it is written. That input is called `filestream`. The file is `host-baseline.tsv` from Lecture 6: a header and 60 rows.

CLIENT-01

```
cd /home/ubuntu/data
unzip -o cis3370-generate-bundle.zip
mkdir -p lab
cp cis3370-generate-bundle/host-baseline.tsv lab/
wc -l lab/host-baseline.tsv
```

```
61 lab/host-baseline.tsv
```

Keep the original configuration as a reference and write a short one:

```
cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.original
cat > /etc/filebeat/filebeat.yml <<'EOF'
filebeat.inputs:
  - type: filestream
    id: host-baseline
    paths:
      - /home/ubuntu/data/lab/host-baseline.tsv
    fields:
      lab: simple-file

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
  reload.enabled: false

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]
  username: "beats"
  password: "${ES_PWD}"
EOF
```

SETTING	WHAT IT DOES
`id: host-baseline`	Names this input. The registry tracks how far it has read by this name.
`fields.lab`	A label on every document from this input, so you can find them
`password: "${ES_PWD}"`	Read from the keystore when Filebeat starts. The file never holds the secret.

Test, then set up as the admin

```
filebeat test config
filebeat test output
```

```
Config OK
...
  talk to server... OK
  version: 9.5.3
```

Run setup as `beats` first, and watch the role do its job:

```
filebeat setup --index-management
```

```
Exiting: ... got 403 from elasticsearch: ... action [cluster:admin/ilm/put] is unauthorized
for user [beats] with effective roles [filebeat_publisher], this action is granted by the
cluster privileges [manage_ilm,manage,all]
```

Now run it with the admin key. The `-E` flags change the settings for this one command. `null` removes the username and password, because Filebeat refuses a username and a key together. The single quotes keep the shell from touching `${ADMIN_KEY}`, so Filebeat reads it from the keystore.

```
filebeat setup --index-management \
  -E output.elasticsearch.username=null \
  -E output.elasticsearch.password=null \
  -E 'output.elasticsearch.api_key=${ADMIN_KEY}'
```

```
Index setup finished.
```

Start Filebeat and count three ways

```
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

Wait about 20 seconds. Counting needs read access, so use your key, not `beats`.

CONSOLE

```
GET filebeat-*/_count?q=fields.lab:simple-file
```

CURL

HOST

```
curl -s -H "$KEYFILE" "$ES/filebeat-*/_count?q=fields.lab:simple-file"
```

```
{"count":61, ...}
```

61 lines, 61 documents. Now the streaming part.

CLIENT-01

```
echo "client-01 appended this line" >> /home/ubuntu/data/lab/host-baseline.tsv
```

Filebeat looks for changes every 10 seconds. Count again after about 20 seconds: 62. Nobody restarted anything. Filebeat remembers where it stopped reading; that memory is the registry, in `/var/lib/filebeat/registry`.

If the count stays at 0. With `create_doc` alone, Filebeat read all 61 lines and dropped every one. Its log said only `Failed to index 61 events in last 10s: events were dropped!`. The reason was in `/var/log/filebeat/filebeat-events-data-*.ndjson`:

```
(status=403): action [indices:admin/mapping/auto_put] is unauthorized for user [beats]
```

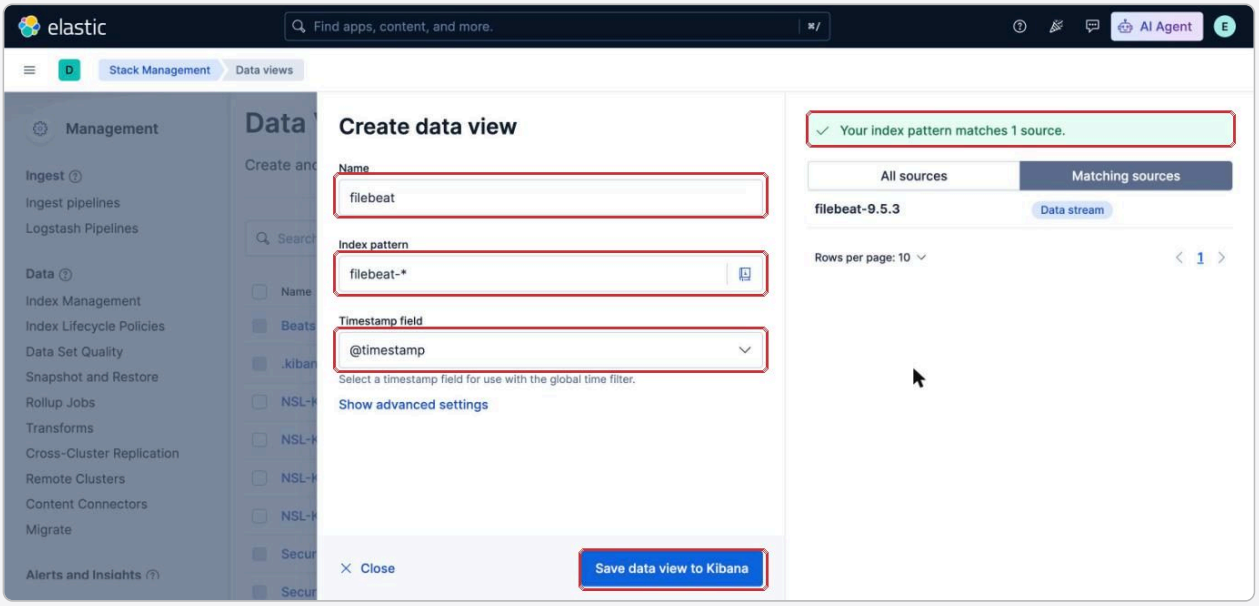
`fields.lab` was a new field, and adding a field needs `auto_configure`. Fix the role, then make Filebeat forget it read the file:

```
pkill filebeat
rm -rf /var/lib/filebeat/registry
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
```

See it in Discover

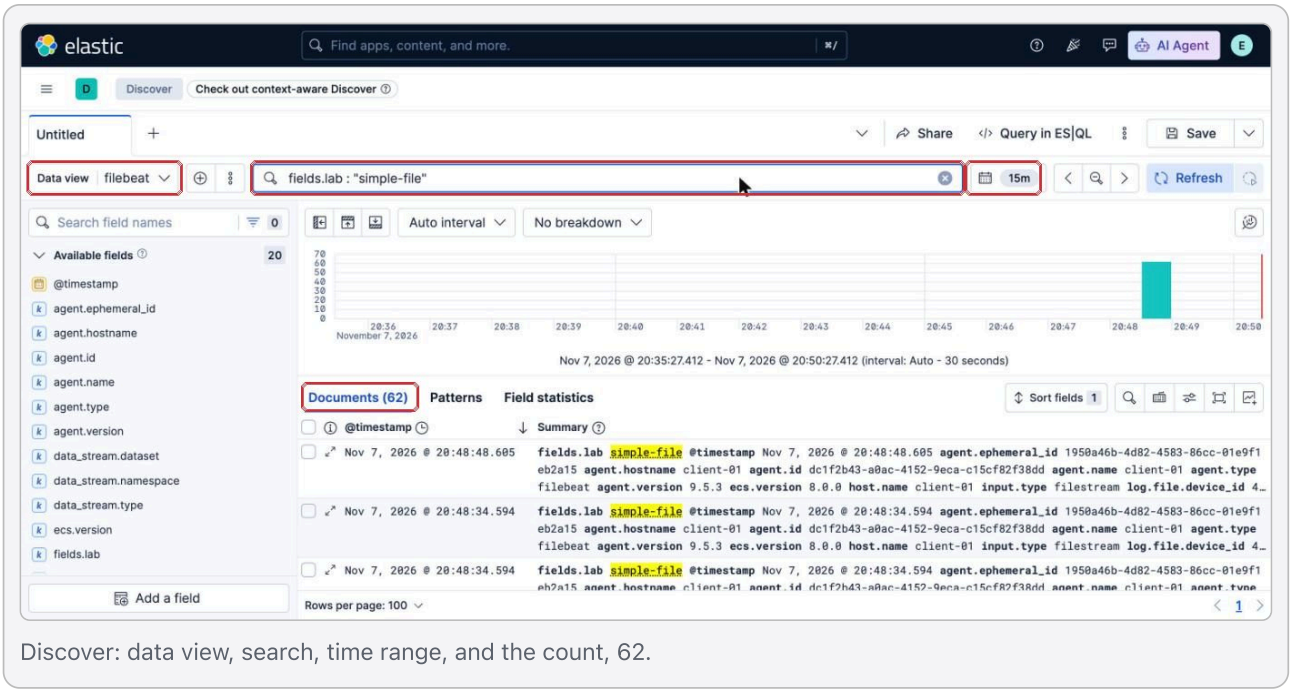
KIBANA UI

Stack Management → Data Views → Create data view. Name `filebeat`, index pattern `filebeat-*`, timestamp field `@timestamp`. Kibana adds a `*` while you type; check the field says exactly `filebeat-*`. The right side shows the match, `filebeat-9.5.3`, a data stream.



Name, index pattern, timestamp field, and the matching data stream on the right.

Open **Discover**, pick the `filebeat` data view, and search `fields.lab : "simple-file"`.



Discover: data view, search, time range, and the count, 62.

Each `message` is one whole line of the file, tabs and all, and `@timestamp` is when Filebeat read the line, not the time written in it. A module fixes both, which is the next section.

7 The system module

A module is a packaged answer to "how do I read this kind of log?". It knows where the files are, and ships an ingest pipeline that splits each line into named fields. The `system` module reads `syslog`, general messages, and `auth.log`, logins.

Give client-01 something to log

`CLIENT-01` A container has no logging service and no SSH server. Install both and start them by hand. The `sed` line turns off kernel logging, which a container may not read.

```
apt-get install -y rsyslog openssh-server
sed -i 's/^module(load="imklog"/#&/' /etc/rsyslog.conf
rsyslogd
mkdir -p /run/sshd && /usr/sbin/sshd
sleep 1; ls /var/log/syslog /var/log/auth.log
```

Enable, configure, test

```
filebeat modules enable system
cat > /etc/filebeat/modules.d/system.yml <<'EOF'
- module: system
  syslog:
    enabled: true
    var.paths: ["/var/log/syslog"]
  auth:
    enabled: true
    var.paths: ["/var/log/auth.log"]
EOF
filebeat test config
filebeat modules list | head -3
```

```
Enabled system
Config OK
Enabled:
system
```

Load the module's ingest pipelines with the admin key. `--force-enable-module-filesets` is needed because with `--modules`, setup ignores `modules.d`; without it the answer is `module system is configured but has no enabled filesets`.

```
filebeat setup --pipelines --modules system --force-enable-module-filesets \
  -E output.elasticsearch.username=null \
  -E output.elasticsearch.password=null \
  -E 'output.elasticsearch.api_key=${ADMIN_KEY}'
pkill filebeat
nohup filebeat -e > /var/log/filebeat-run.log 2>&1 &
logger -t cis3370 "hello from client-01 through syslog"
```

```
Loaded Ingest pipelines
```

Make some evidence from the other machine

`HOST` `docker compose exec zeek-lab bash`, then in `ZEEK-LAB` :

```
apt-get update
apt-get install -y openssh-client
for u in admin root test; do
  ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
    $u@client-01 true
done
```

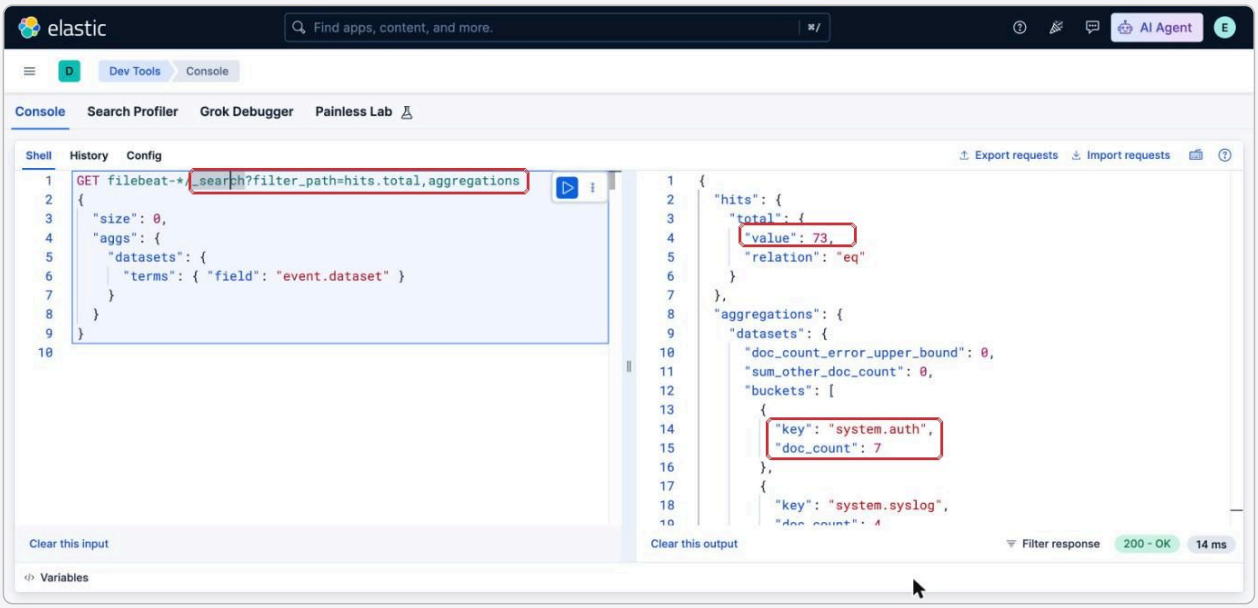
```
admin@client-01: Permission denied (publickey,password).
root@client-01: Permission denied (publickey,password).
test@client-01: Permission denied (publickey,password).
```

Validate three ways

CONSOLE

```
GET filebeat-*/_search?filter_path=hits.total,aggregations
{
  "size": 0,
  "aggs": {
    "datasets": {
      "terms": { "field": "event.dataset" }
    }
  }
}
```

`filter_path` trims the answer to the parts you asked about.



73 documents: system.auth 7 and system.syslog 4, plus the 62 from section 6.

CURL

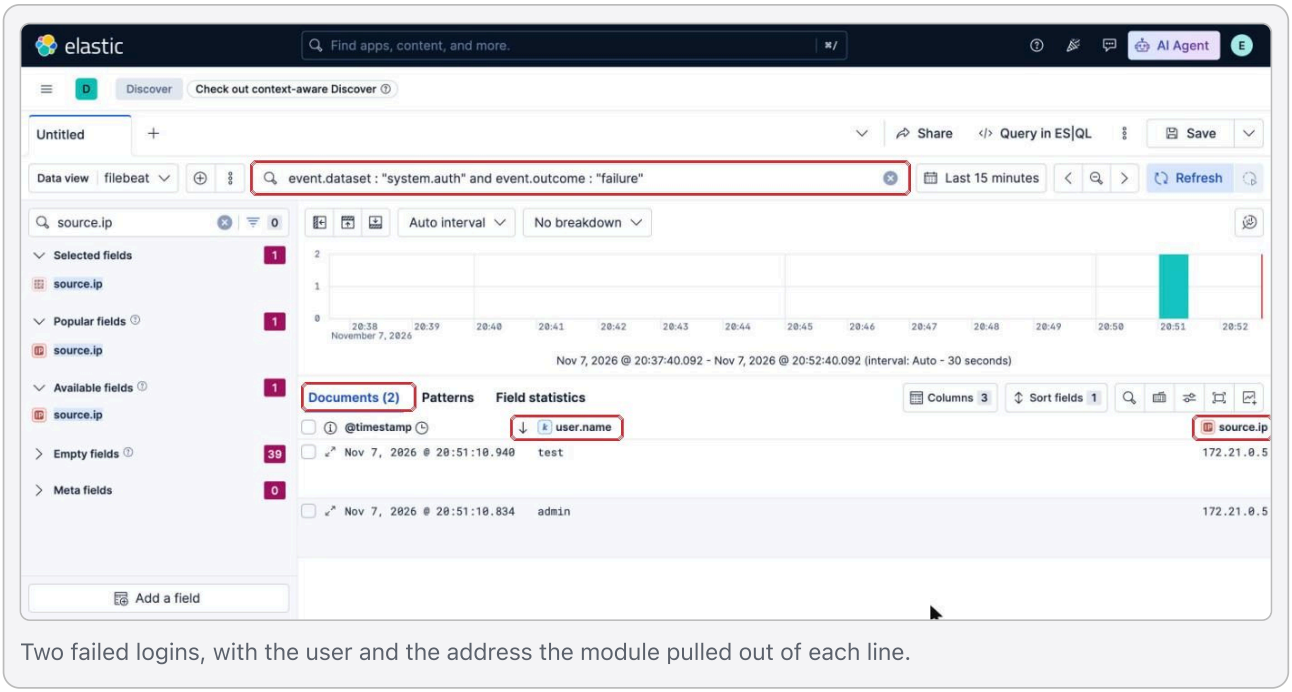
HOST

```
curl -s -H "$KEYFILE" -G "$ES/filebeat-*/_search" \
  -d q=system.auth.ssh.event:Invalid \
  -d _source=user.name,source.ip,event.outcome
```

Two hits: `admin` and `test`, each with `source.ip` set to `zeek-lab`'s address and `event.outcome: failure`.

DISCOVER

Search `event.dataset : "system.auth" and event.outcome : "failure"`, then add `user.name` and `source.ip` as columns with the plus button next to each field name.



Two failed logins, with the user and the address the module pulled out of each line.

The pipeline understood the newer OpenSSH process name, `sshd-session`, with no change. `root` is missing from the failures list: the server called it an "authenticating user", not an "Invalid user", because root exists.

Remove the admin key from client-01

`CLIENT-01` Setup is done. The machine keeps only what it needs to publish.

```
filebeat keystore remove ADMIN_KEY
filebeat keystore list
```

```
successfully removed key: ADMIN_KEY
ES_PWD
```

8 Questions

QUESTION 1

Setup as `beats` was refused. Name the privilege the error said was missing, and explain why a shipper should never hold it.

QUESTION 2

An attacker owns `client-01` and reads `ES_PWD` out of the keystore. List what they can do to Elasticsearch, and what they cannot, using your status codes as evidence.

QUESTION 3

Why can an API key be revoked without affecting anything else, while changing a password cannot? Give one situation where that matters.

QUESTION 4

The raw file's `@timestamp` is when Filebeat read the line. The system module's is when the event happened. Which one would you trust in an investigation, and why?

QUESTION 5

`admin` and `test` are "Invalid user", but `root` is not. What did the server just tell the person guessing, and what does it tell you?

QUESTION 6

You edited `/etc/filebeat/filebeat.yml` inside the container. What happens to it after `docker compose down`, and how would you keep it?

9 What to hand in

1. The output of the two `docker compose ps` commands and of the read-only test.
2. A screenshot of your API keys list showing your key with an expiry, and the output of `_security/_authenticate` through your header file.
3. The four status codes from the beats checks.
4. Your counts: lines and documents for the simple file before and after the append, and the `system.auth` and `system.syslog` counts.
5. Your Discover screenshot of the failed logins.
6. Your answers to the six questions.

10 Start over

`CLIENT-01`

```
pkill filebeat
rm -rf /var/lib/filebeat/registry
```

`HOST` Remove what this lab created. The Console versions are the same paths with the same methods.

```
curl -s -H "$KEYFILE" -X DELETE "$ES/_data_stream/filebeat-9.5.3"
curl -s -H "$KEYFILE" -X DELETE "$ES/_index_template/filebeat-9.5.3"
curl -s -H "$KEYFILE" -X DELETE "$ES/_ilm/policy/filebeat"
curl -s -H "$KEYFILE" -X DELETE "$ES/_ingest/pipeline/filebeat-9.5.3-*"
curl -s -H "$KEYFILE" -X DELETE "$ES/_security/user/beats"
curl -s -H "$KEYFILE" -X DELETE "$ES/_security/role/filebeat_publisher"
```

Labs B and C build on this one, so do this only when you are finished with all three.

A The class docker-elk compose file

This replaces the `docker-compose.yml` of a docker-elk checkout at commit `a41b72b`, the 9.5.3 release. Run `mkdir -p zeek-lab/data client-01/data` before the first `docker compose up`.

```
# The class docker-elk environment in one file.
#
# This replaces the docker-compose.yml of a docker-elk checkout at the
# "Update to v9.5.3" commit (a41b72b). It is docker-elk's own file with two
# changes: more memory for Elasticsearch and Logstash, and two Ubuntu lab
# containers, zeek-lab and client-01, on the same network as the stack.
# The README.md beside this file has the steps.

services:

  # The 'setup' service runs a one-off script which initializes users inside
  # Elasticsearch, such as 'logstash_internal' and 'kibana_system', with the
  # values of the passwords defined in the '.env' file. It also creates the
  # roles required by some of these users.
  #
  # This task only needs to be performed once, during the *initial* startup of
  # the stack. Any subsequent run will reset the passwords of existing users to
  # the values defined inside the '.env' file, and the built-in roles to their
  # default permissions.
  #
  # By default, it is excluded from the services started by 'docker compose up'
  # due to the non-default profile it belongs to. To run it, either provide the
  # '--profile=setup' CLI flag to Compose commands, or "up" the service by name
  # such as 'docker compose up setup'.
  setup:
    profiles:
      - setup
    build:
      context: setup/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    init: true
    volumes:
      - ./setup/entrypoint.sh:/entrypoint.sh:ro,Z
      - ./setup/lib.sh:/lib.sh:ro,Z
      - ./setup/roles:/roles:ro,Z
    environment:
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD:-}
      LOGSTASH_INTERNAL_PASSWORD: ${LOGSTASH_INTERNAL_PASSWORD:-}
      KIBANA_SYSTEM_PASSWORD: ${KIBANA_SYSTEM_PASSWORD:-}
      METRICBEAT_INTERNAL_PASSWORD: ${METRICBEAT_INTERNAL_PASSWORD:-}
      FILEBEAT_INTERNAL_PASSWORD: ${FILEBEAT_INTERNAL_PASSWORD:-}
      HEARTBEAT_INTERNAL_PASSWORD: ${HEARTBEAT_INTERNAL_PASSWORD:-}
      MONITORING_INTERNAL_PASSWORD: ${MONITORING_INTERNAL_PASSWORD:-}
      BEATS_SYSTEM_PASSWORD: ${BEATS_SYSTEM_PASSWORD:-}
    network_mode: service:elasticsearch
    depends_on:
      - elasticsearch

  # The 'kibana-genkeys' service runs a one-off command which generates and
  # prints encryption keys for Kibana.
  #
  # By default, it is excluded from the services started by 'docker compose up'
  # due to the non-default profile it belongs to. To run it, either provide the
  # '--profile=setup' CLI flag to Compose commands, or "up" the service by name
  # such as 'docker compose up kibana-genkeys'.
  kibana-genkeys:
    profiles:
```

```
      - setup
    build:
      context: kibana/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    command:
      - bin/kibana-encryption-keys
      - generate
    network_mode: none

  elasticsearch:
    build:
      context: elasticsearch/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      -
./elasticsearch/config/elasticsearch.yml:/usr/share/elasticsearch/config/elasticsearch.yml:ro,Z
      - elasticsearch:/usr/share/elasticsearch/data:Z
    ports:
      - 9200:9200
      - 9300:9300
    environment:
      node.name: elasticsearch
      ES_JAVA_OPTS: -Xms2048m -Xmx2048m
      # Bootstrap password.
      # Used to initialize the keystore during the initial startup of
      # Elasticsearch. Ignored on subsequent runs.
      ELASTIC_PASSWORD: ${ELASTIC_PASSWORD:-}
      # Use single node discovery in order to disable production mode and avoid bootstrap
checks.
      # see: https://www.elastic.co/docs/deploy-manage/deploy/self-managed/bootstrap-checks
      discovery.type: single-node
    networks:
      - elk
    restart: unless-stopped

  logstash:
    build:
      context: logstash/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      - ./logstash/config/logstash.yml:/usr/share/logstash/config/logstash.yml:ro,Z
      - ./logstash/pipeline:/usr/share/logstash/pipeline:ro,Z
    ports:
      - 5044:5044
      - 50000:50000/tcp
      - 50000:50000/udp
      - 9600:9600
    environment:
      LS_JAVA_OPTS: -Xms1024m -Xmx1024m
      LOGSTASH_INTERNAL_PASSWORD: ${LOGSTASH_INTERNAL_PASSWORD:-}
    networks:
      - elk
    depends_on:
      - elasticsearch
    restart: unless-stopped
```

```
  kibana:
    build:
      context: kibana/
      args:
        ELASTIC_VERSION: ${ELASTIC_VERSION}
    volumes:
      - ./kibana/config/kibana.yml:/usr/share/kibana/config/kibana.yml:ro,Z
    ports:
      - 5601:5601
    environment:
      KIBANA_SYSTEM_PASSWORD: ${KIBANA_SYSTEM_PASSWORD:-}
    networks:
      - elk
    depends_on:
      - elasticsearch
    restart: unless-stopped

  # Two lab machines on the same 'elk' network as the stack. Both are plain
  # Ubuntu 26.04 that stay running so you can open a shell in them:
  #
  #   docker compose exec zeek-lab bash
  #   docker compose exec client-01 bash
  #
  # Each one mounts a folder from this repository at /home/ubuntu/data, so files
  # you put there from your computer appear inside the container, and files the
  # container writes there (captures, logs) appear on your computer. They also
  # survive 'docker compose down -v', because they are folders, not volumes.
  #
  # The other containers reach them by name: 'zeek-lab', 'client-01',
  # 'elasticsearch', 'logstash', 'kibana'. NET_ADMIN and NET_RAW let tcpdump
  # capture and nmap send raw packets.
  zeek-lab:
    image: ubuntu:26.04
    hostname: zeek-lab
    command: sleep infinity
    init: true
    working_dir: /home/ubuntu
    volumes:
      - ./zeek-lab/data:/home/ubuntu/data:Z
    cap_add:
      - NET_ADMIN
      - NET_RAW
    networks:
      - elk
    restart: unless-stopped

  client-01:
    image: ubuntu:26.04
    hostname: client-01
    command: sleep infinity
    init: true
    working_dir: /home/ubuntu
    volumes:
      - ./client-01/data:/home/ubuntu/data:Z
    cap_add:
      - NET_ADMIN
      - NET_RAW
    networks:
      - elk
```

```
    restart: unless-stopped

networks:
  elk:
    driver: bridge

volumes:
  elasticsearch:
```

Every number and screenshot on this page comes from running these steps on the class stack: Elasticsearch, Kibana and Filebeat 9.5.3 on Ubuntu 26.04 containers. Addresses, IDs and times differ on your machine. Counts from the bundle should not; if one does, find out why.